HTML & CSS

Phase 2 — Intermediate

Box Model · Display · Flexbox · CSS Grid · Positioning · Pseudo-classes · Responsive Design

9. Box Model	*10. Display Property*	*11. Flexbox*	*12. CSS Grid*
13. Positioning	*14. Pseudo-classes & Pseudo-elements*	*15. Responsive Design & Media Queries*	

PHASE 2

Intermediate

Box Model · Display · Flexbox · Grid · Positioning · Pseudo · Responsive

9 The Box Model — Everything Is a Box

The single most important CSS layout concept: **every element on a webpage is a rectangular box**. Text, images, buttons, divs — all boxes. Each has 4 layers from inside to outside: **Content** (actual text/image), **Padding** (space inside the border), **Border** (the edge), and **Margin** (space outside, pushing other elements away).

Think of it like shipping a fragile item: the item = content, bubble wrap = padding, the box = border, space between boxes in the truck = margin.

> ■ REAL WORLD
> Real World: You set width: 33% on 3 cards in a flex row. You add padding: 20px and the cards suddenly wrap to the next line. Why? Default box-sizing adds padding ON TOP of your width. The fix: box-sizing: border-box.

```
/* ■■■ ALWAYS include this reset at the top of every project ■■■ */
*, *::before, *::after { box-sizing: border-box; }

/* Without border-box (default content-box — broken) */
.card { width: 300px; padding: 20px; border: 2px solid gray; }
/* Actual width = 300 + 20 + 20 + 2 + 2 = 344px SURPRISE! */

/* With border-box (correct — predictable) */
.card { box-sizing: border-box; width: 300px; padding: 20px; border: 2px solid gray; }
/* Actual width = 300px — padding & border are INSIDE the width */

/* ■■■ Margin Shorthand ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
margin: 20px; /* All 4 sides */
margin: 10px 20px; /* Top/Bottom | Left/Right */
margin: 10px 20px 15px 5px; /* Top | Right | Bottom | Left */
margin: 0 auto; /* Center element horizontally */

/* ■■■ Margin Collapse (important gotcha) ■■■■■■■■■■■■■■■■■■■■■■ */
.heading { margin-bottom: 32px; }
.paragraph { margin-top: 16px; }
/* Gap = 32px (the larger value), NOT 48px — they collapse! */
/* Only happens vertically. Does NOT happen in flex/grid. */
```

	content-box (default)	border-box (recommended)
width: 300px + padding: 20px	Renders as 340px	Renders as 300px
Predictable sizing?	No — surprises everywhere	Yes — always as declared
How padding affects it	Adds to total width	Shrinks content area instead
Should you use it?	Never	Always — set as global reset

Q: What is the CSS Box Model?

A: Every HTML element is a rectangular box with 4 layers. From inside out: content (text/image), padding (space between content and border), border (the element edge), and margin (space outside pushing other elements away). Understanding these layers is fundamental to controlling all layout and spacing in CSS.

Q: What is margin collapse and when does it NOT happen?

A: Margin collapse is when vertical margins of adjacent block elements merge into one margin equal to the LARGER value, not the sum. It happens between block-level siblings and parent/first-child with no separating border or padding. It does NOT happen with flex or grid children, elements with overflow other than visible, absolute/fixed positioned elements, or between inline-block elements.

10 The display Property — How Elements Behave

The **display property** is one of the most powerful in CSS. It controls how an element participates in layout — does it start on a new line? Does it respect width and height? Is it a layout container for its children?

```
/* display: block ■■■
```

Q: What is the difference between inline and inline-block?

A: Both flow inline — they sit beside other elements on the same line. The difference: inline ignores width, height, and vertical margin/padding — size is driven purely by content. inline-block flows inline BUT fully respects width, height, and all four sides of margin and padding. Use inline-block when you need sized elements that still flow with surrounding text or other inline content.

Q: What is the default display value of div vs span?

A: div is display: block by default — takes full width and starts on a new line. span is display: inline — flows within text, only takes content width. This is why div is used for layout sections and span for styling text portions. Both can be overridden with CSS display property.

11 Flexbox — One-Dimensional Layout

Flexbox solves **one-dimensional layout** — arranging items in a row OR a column. The parent is the **flex container** (has display: flex). The direct children are **flex items**. Two axes always exist: **main axis** (direction of flow) and **cross axis** (perpendicular).

> ■ **GOTCHA**
> justify-content works on the MAIN axis. align-items works on the CROSS axis. When flex-direction is column, these swap. Neither is inherently horizontal or vertical — it always depends on flex-direction.

```
/* ■■■ CONTAINER PROPERTIES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
.container {
display: flex;
flex-direction: row; /* row | column | row-reverse | column-reverse */
justify-content: space-between; /* Main axis: start|end|center|space-between|space-evenly */
align-items: center; /* Cross axis: stretch|start|end|center|baseline */
flex-wrap: wrap; /* Allow items to wrap to next line */
gap: 16px; /* Space between items */
}

/* ■■■ ITEM PROPERTIES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* flex: grow shrink basis */
.sidebar { flex: 0 0 280px; } /* Fixed 280px — never grows or shrinks */
.main { flex: 1 1 0; } /* Takes all remaining space */
.logo { flex: none; } /* = flex: 0 0 auto */
.special { align-self: flex-end; } /* Override alignment for this item only */

/* ■■■ REAL PATTERNS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

/* 1. Navbar */
.navbar { display:flex; justify-content:space-between; align-items:center; height:64px; padding:0 24px; }

/* 2. Perfect centering */
.hero { display:flex; justify-content:center; align-items:center; min-height:100vh; }

/* 3. Sticky footer */
body { display:flex; flex-direction:column; min-height:100vh; }
main { flex: 1; } /* Takes leftover height — pushes footer to bottom */

/* 4. Card footer always at bottom */
.card { display:flex; flex-direction:column; }
.card__footer { margin-top: auto; } /* auto margin pushes to end */

/* 5. Equal-width items */
.tabs > * { flex: 1; } /* Every child takes equal share */
```

flex shorthand	Expands to	Behavior
flex: 1	flex: 1 1 0%	Grow & shrink; all siblings share space equally from 0 base
flex: auto	flex: 1 1 auto	Grow & shrink; larger content gets proportionally more space
flex: none	flex: 0 0 auto	Rigid — never grows or shrinks from content size
flex: 0 0 300px	Same	Always exactly 300px — ignores all extra space

Q: What is the difference between justify-content and align-items?

A: justify-content aligns items along the MAIN axis — the direction of flex-direction. align-items aligns items along the CROSS axis — perpendicular. In flex-direction: row, justify-content is horizontal and align-items is vertical. In flex-direction: column, they swap completely. They are axis-relative, not direction-specific.

Q: What is the difference between flex: 1 and flex: auto?

A: flex: 1 expands to flex: 1 1 0% — flex-basis is 0, so all items start from zero and extra space is distributed equally. Siblings get equal width. flex: auto expands to flex: 1 1 auto — basis is content size, so space is distributed AFTER accounting for each item's natural width. Larger content gets more space.

12 CSS Grid — Two-Dimensional Layout

Where Flexbox handles one direction, **CSS Grid handles two dimensions simultaneously** — rows AND columns at the same time. Perfect for page-level layouts and components where items must align across both axes.

> ■ **REAL WORLD**
> Real World: An admin dashboard — sidebar, topnav, main content, right panel. Grid does this in ONE parent declaration. Flexbox would need 3-4 levels of nesting to achieve the same result.

```
/* ■■■ DEFINING THE GRID ■■■
```

```
/* auto-fill: fill row with as many 280px+ columns as fit */
/* Result: 4 cols on desktop, 2 on tablet, 1 on mobile */
gap: 24px;
}
```

	Flexbox	CSS Grid
Dimensions	1D — row OR column	2D — rows AND columns simultaneously
Best for	Navbars, card rows, form fields	Dashboards, page layouts, card grids
Layout driven by	Content (items define space)	Structure (tracks define space)
Overlapping items	Difficult	Easy — place items on same cell
Use together?	Grid for page skeleton	Flex inside grid components

Q: What is the difference between auto-fill and auto-fit in CSS Grid?

A: Both fill the row with as many columns as fit given a minimum size. The difference: with fewer items than available columns, auto-fill creates empty ghost column tracks to maintain structure. auto-fit collapses empty tracks to zero, letting existing items grow and fill the space. Use auto-fill for consistent grids, auto-fit when items should stretch.

Q: When would you use Grid over Flexbox?

A: Use Grid when items must align across both rows AND columns simultaneously — dashboards, photo galleries, page skeletons. Use Flexbox for one-directional flow where content drives sizing — navbars, button groups, form fields. In practice use both: Grid for the outer page structure, Flexbox inside components.

13 CSS Positioning — Full Control of Placement

The **position property** changes how an element is placed and changes what top/left/right/bottom do. Most values take the element out of **normal flow** — meaning other elements behave as if it doesn't exist.

```
/* static — default. Offsets do nothing. Element in normal flow. */
.box { position: static; } /* Same as not writing position at all */

/* relative — stays in flow. Offset from its OWN position. */
/* CRITICAL: creates positioning context for absolute children */
.card { position: relative; top: 10px; left: 20px; }
/* Original space is still reserved — visible gap appears */

/* absolute — REMOVED from flow. Other elements ignore it. */
/* Positioned relative to nearest POSITIONED ancestor. */
.card { position: relative; } /* Reference point */
.badge {
position: absolute;
top: -8px; /* 8px above card's top edge */
right: -8px; /* 8px outside card's right edge */
/* Without relative on .card: positions to entire page! */
}

/* fixed — REMOVED from flow. Positions to VIEWPORT always. */
/* Stays in place when user scrolls. */
.navbar { position: fixed; top: 0; left: 0; right: 0; z-index: 100; }
body { padding-top: 64px; } /* Push content below fixed navbar */

/* sticky — relative until threshold, then fixed-like. */
/* REQUIRES top/bottom/left/right — without it: does nothing */
.table-header { position: sticky; top: 0; z-index: 10; background: white; }

/* inset: shorthand for top + right + bottom + left */
.overlay { position: fixed; inset: 0; background: rgba(0,0,0,0.5); }

/* z-index: stacking order — ONLY works on positioned elements */
.modal { position: fixed; inset: 0; z-index: 1000; }
.tooltip { position: absolute; z-index: 500; }
.navbar { position: fixed; z-index: 100; }
```

Value	In flow?	Positioned relative to	Scrolls with page?
static (default)	Yes	— (offsets do nothing)	Yes
relative	Yes	Its own original position	Yes
absolute	No	Nearest positioned ancestor	With ancestor
fixed	No	Viewport — always	No — stays fixed
sticky	Yes (until threshold)	Scroll container	Partially

Q: Why is position: absolute not working as expected?

A: The most common reason: no positioned ancestor. absolute positions relative to the nearest ancestor with position other than static. If none exists, it positions to the html element — not the parent you intended. Fix: add position: relative to the

intended parent container. No offset needed — just position: relative is enough to establish the reference point.

Q: Why is position: sticky not working?

A: Three common reasons: (1) No top/left/right/bottom value defined — sticky needs a threshold to activate. (2) A parent has overflow: hidden, auto, or scroll — creates a new scroll container and the element cannot escape it to stick to the viewport. (3) The parent element is not tall enough — the sticky element needs room to scroll within its container before sticking.

14 Pseudo-classes & Pseudo-elements

Pseudo-classes target elements based on their state or position — is the user hovering? Is it the first child? Is the input valid? **Pseudo-elements** target a specific PART of an element — before/after its content, the first line, or the placeholder text.

```
/* ■■■ PSEUDO-CLASSES (:) — state or position ■■■■■■■■■■■■■■■ */

/* User interaction */
.btn:hover { background: #4f46e5; transform: translateY(-1px); }
.btn:active { transform: scale(0.98); }
.btn:focus { outline: 2px solid #6366f1; outline-offset: 2px; }

/* Form states */
input:focus { border-color: #6366f1; box-shadow: 0 0 0 3px rgba(99,102,241,0.2); }
input:disabled { opacity: 0.5; cursor: not-allowed; }
input:valid { border-color: #059669; }
input:invalid { border-color: #dc2626; }

/* Structural / positional */
li:first-child { font-weight: bold; }
li:last-child { border-bottom: none; }
li:nth-child(2) { background: #f1f5f9; }
li:nth-child(odd) { background: #f8fafc; } /* 1st, 3rd, 5th... */
li:nth-child(even) { background: white; } /* 2nd, 4th, 6th... */

/* :not() — exclude matching elements */
li:not(:last-child) { border-bottom: 1px solid #e2e8f0; }
.card:not(.featured) { opacity: 0.8; }

/* :has() — parent selector (modern CSS game changer) */
.card:has(img) { padding: 0; }
form:has(input:invalid) { border: 2px solid red; }

/* ■■■ PSEUDO-ELEMENTS (::) — parts of elements ■■■■■■■■■■■■ */

/* ::before and ::after — virtual elements around content */
/* ALWAYS requires content: '' property — even if empty! */

/* Required field asterisk */
.label--required::after { content: ' *'; color: #dc2626; }

/* Custom bullet */
li::before { content: '->'; color: #6366f1; margin-right: 8px; }

/* Decorative underline on heading */
h2::after {
content: ''; /* Empty — purely decorative */
```

```
display: block;
width: 48px; height: 3px;
background: #6366f1;
margin-top: 8px;
border-radius: 2px;
}

/* ::placeholder — style ghost text in inputs */
input::placeholder { color: #94a3b8; font-style: italic; }

/* ::selection — style text user has highlighted */
::selection { background: #6366f1; color: white; }
```

	Pseudo-class (:)	Pseudo-element (::)
Targets	Element in a certain state or position	A specific PART of an element
Syntax	Single colon :hover, :focus	Double colon ::before, ::after
Examples	:hover, :focus, :nth-child(), :not()	::before, ::after, ::placeholder
Adds content?	No	Yes (::before, ::after with content: '')
How many per element?	Multiple allowed	One ::before + one ::after

Q: What is required for ::before and ::after to work?

A: The content property — it is mandatory. Even if you want a purely decorative element with no visible text, you must write content: '' (empty string). Without it, the pseudo-element does not render at all. It is inline by default, so you usually also need display: block to apply width/height. It is NOT part of the real DOM — it cannot be selected or manipulated with JavaScript.

Q: What is the difference between :nth-child() and :nth-of-type()?

A: :nth-child(2) selects an element if it is the 2nd child of its parent regardless of tag type. :nth-of-type(2) selects the 2nd element of that specific tag among siblings. Example: in a div containing h2, p, p — p:nth-child(2) selects the first p because it is the 2nd child. p:nth-of-type(2) reliably selects the second p regardless of any other sibling element types.

15 Responsive Design & Media Queries

Responsive design means your webpage looks and works correctly on ALL screen sizes. The tools: **Media Queries** (apply CSS only when conditions are met), **fluid units** (%, vw, rem instead of fixed px), and **flexible layouts** (Flexbox wrap, Grid auto-fill).

> ■ **TIP**
> Industry standard: always write mobile-first. Base CSS for mobile, then min-width media queries to enhance for larger screens. Most web traffic is mobile.

```
/* ■■■ BASIC SYNTAX ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@media (condition) {
/* CSS that ONLY applies when condition is true */
}

/* ■■■ MOBILE FIRST (recommended) ■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* Base styles — for mobile (written OUTSIDE media queries) */
.card { padding: 12px; font-size: 14px; }
.card-grid { grid-template-columns: 1fr; } /* 1 column */
.nav { flex-direction: column; gap: 8px; }

/* Tablet: 640px and up */
@media (min-width: 640px) {
.card { padding: 20px; }
.card-grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop: 1024px and up */
@media (min-width: 1024px) {
.card { padding: 24px; font-size: 16px; }
.card-grid { grid-template-columns: repeat(3, 1fr); }
.nav { flex-direction: row; }
}

/* Large desktop: 1280px and up */
@media (min-width: 1280px) {
.container { max-width: 1200px; margin: 0 auto; }
}

/* ■■■ OTHER USEFUL MEDIA QUERIES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@media (prefers-color-scheme: dark) {
body { background: #0f172a; color: #f1f5f9; }
}

@media (prefers-reduced-motion: reduce) {
*, *::before, *::after {
animation-duration: 0.01ms !important;
transition-duration: 0.01ms !important;
}
}

@media print {
.navbar, .sidebar { display: none; }
}

/* ■■■ FLUID UNITS — responsive without media queries ■■■■■■■■ */
```

```
h1 { font-size: clamp(1.5rem, 5vw, 3.5rem); }
/* clamp(min, preferred, max) — fluid and capped automatically */

.container { width: clamp(320px, 90%, 1200px); }
```

Approach	Direction	Uses	Recommended?
Mobile-first	Small to Large	min-width media queries	Yes — industry standard
Desktop-first	Large to Small	max-width media queries	No — avoid

Unit	Relative to	Best used for
px	Nothing (absolute)	Borders, shadows, fixed small values
rem	Root font size (16px default)	Font sizes, consistent spacing across page
em	Current element's font size	Component-internal spacing, icon sizes
%	Parent element dimensions	Widths in fluid layouts
vw / vh	Viewport width / height	Hero sections, full-screen layouts
clamp()	Combination of all	Fluid typography, responsive containers

Q: What is mobile-first design and why is it recommended?

A: Mobile-first means writing base CSS for the smallest screen, then adding min-width media queries to progressively enhance for larger screens. It is recommended because most web traffic is mobile, it forces you to prioritize essential content, mobile-first CSS loads faster on mobile since larger-screen media queries are skipped, and it aligns with progressive enhancement — start simple, add complexity when space allows.

Q: What is the difference between min-width and max-width in media queries?

A: min-width means 'apply these styles when screen is AT LEAST this wide' — used in mobile-first to add styles as screens get bigger. max-width means 'apply these styles when screen is AT MOST this wide' — used in desktop-first to remove styles for smaller screens. min-width is preferred because it encourages simpler base styles and adds complexity progressively only when the extra space is actually available.